Set in Java

Last Updated : 07 Aug, 2025

In Java, the Set interface is a part of the Java Collection Framework, located in the java.util package. It represents a collection of unique elements, meaning it does not allow duplicate values.

Key Features of Set

- **No duplicates:** Set does not allow duplicate elements; each item must be unique.
- **No guaranteed order:** Elements in a Set are not stored or retrieved in any defined order.
- **Ordering exceptions:** LinkedHashSet maintains insertion order, while TreeSet keeps elements sorted.
- **One null allowed:** Most Set implementations allow only a single null element.
- **Collection methods inherited:** Set supports standard methods like add(), remove(), contains(), size() and iterator() from the Collection interface.

Example: Java Program to Implementing Set Interface

```
import java.util.HashSet;
import java.util.Set;

public class Geeks {

    public static void main(String args[])
    {
        // Create a Set using HashSet
```

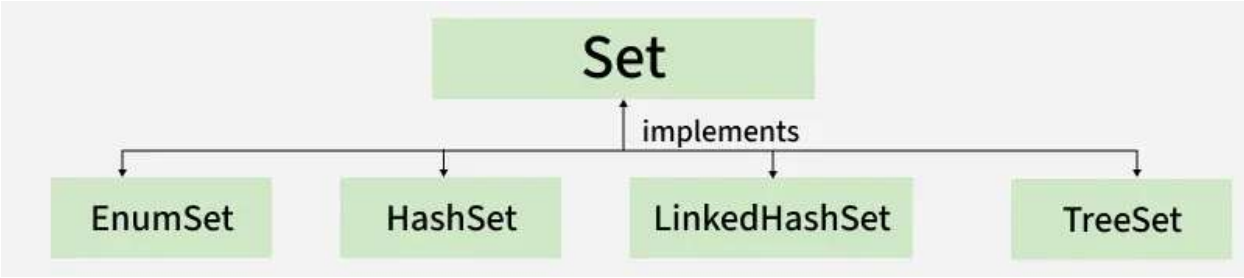
```
        Set<String> s = new HashSet<>();

        // Displaying the Set
        System.out.println("Set Elements: " + s);
    }
}
```

Explanation: In the above example, HashSet will appear as an empty set, as no elements were added. The order of elements in HashSet is not guaranteed, so the elements will be displayed in a random order if any are added.

Hierarchy of Java Set interface

The image below demonstrates the hierarchy of Java Set interface.



Set-Hierarchy

Declaration of Set Interface

The declaration of Set interface is listed below:

public interface Set extends Collection

Creating Set Objects

Since Set is an interface, objects cannot be created of the typeset. We always need a class that extends this list in order to create an object. And also, after the introduction of Generics in Java 1.5, it is possible to restrict the type of object that can be stored in the Set. This type-safe set can be defined as:

// Obj is the type of the object to be stored in Set

Set<Obj> set = new HashSet<Obj> ();

Performing Various Operations on Set

Set interface provides commonly used operations to manage unique elements in a collection. These include:

1. Adding elements
2. Accessing elements
3. Removing elements
4. Iterating elements

Now let us discuss these operations individually as follows:

1. Adding Elements

To add elements to a Set in Java, use the add() method.

Example: This example demonstrates how HashSet stores unique element and does not maintain any insertion order.

```
import java.util.*;

// Main class
class Geeks {

    public static void main(String[] args)
    {
        Set<String> s = new HashSet<String>();

        s.add("B");
        s.add("B");
        s.add("C");
        s.add("A");

        System.out.println(s);
    }
}
```

Output

```
[A, B, C]
```

2. Accessing the Elements

After adding the elements, if we wish to access the elements, we can use inbuilt methods like contains().

Example: This example demonstrates how to check if a specified element exists or not using the contains() method.

```
import java.util.*;

class Geeks {

    public static void main(String[] args)
    {
        Set<String> h = new HashSet<String>();

        h.add("A");
        h.add("B");
        h.add("C");
        h.add("A");

        System.out.println("Set is " + h);

        String s = "D";

        System.out.println("Contains " + s + " " +
h.contains(s));
    }
}
```

Output

```
Set is [A, B, C]
```

```
Contains D false
```

3. Removing Elements

The values can be removed from the Set using the remove() method.

Example: This example demonstrates how to remove element from a HashSet using remove() method

```
import java.util.*;
class Geeks {

    public static void main(String[] args)
    {
        // Declaring object of Set of type String
        Set<String> h = new HashSet<String>();

        // Elements are added using add() method, Custom input
elements
        h.add("A");
        h.add("B");
        h.add("C");
        h.add("B");
        h.add("D");
        h.add("E");

        System.out.println("Initial HashSet " + h);

        // Removing custom element using remove() method
        h.remove("B");
        System.out.println("After removing element " + h);
    }
}
```

Output

```
Initial HashSet [A, B, C, D, E]
After removing element [A, C, D, E]
```

4. Iterating elements

There are various ways to iterate through the Set. The most famous one is to use the enhanced for loop.

Example: This example demonstrates how to iterate through a HashSet.

```
import java.util.*;

class Geeks {

    public static void main(String[] args)
    {
        // Creating object of Set and declaring String type
        Set<String> h = new HashSet<String>();

        // Adding elements to Set using add() method, Custom
input elements
        h.add("A");
        h.add("B");
        h.add("C");
        h.add("B");
        h.add("D");
        h.add("E");

        // Iterating through the Set via for-each loop
        for (String value : h)

            // Printing all the values inside the object
            System.out.print(value + ", ");

        System.out.println();
    }
}
```

Output

```
A, B, C, D, E,
```

Classes that implement the Set interface

1. HashSet: HashSet is a collection class that implements a hash table-based Set, storing elements based on their hashcode without maintaining insertion order and allowing one null element.

2. EnumSet: EnumSet is a specialized Set implementation for use with enum types. It is part of the Java Collections Framework and offers high performance, often faster than HashSet. All elements in an EnumSet must belong to the same enum type, defined at creation time.

3. LinkedHashSet: LinkedHashSet is an ordered version of HashSet that maintains insertion order using a doubly-linked list across all elements.

4. TreeSet: TreeSet is a SortedSet implementation that stores elements in ascending order using a tree structure, ensuring natural ordering or a custom comparator.

Methods of Set Interface

Let us discuss methods present in the Set interface provided below in a tabular format below as follows:

Method	Description
add(element)	Adds element if not already present. Returns true if added.
addAll(collection)	Adds all elements from the given collection.
clear()	Removes all elements from the set.
contains(element)	Checks if the set contains the specified element.
containsAll(collection)	Checks if the set contains all elements from the given collection.

Method	Description
hashCode()	Returns the hash code of the set.
isEmpty()	This method is used to check whether the set is empty or not.
iterator()	This method is used to return the iterator of the set.
remove(element)	Removes the specified element from the set.
removeAll(collection)	Removes all elements in the given collection from the set.
retainAll(collection)	Retains only elements present in the given collection.
size()	Returns the number of elements in the set.
toArray()	This method is used to form an array of the same elements as that of the Set.

Set Interface In Java

Visit Course

Comment

K kartik + Follow 158

Article Tags : Java Java-Collections Java - util package java-set

Explore

Java Basics

OOP & Interfaces

Collections

Exception Handling

Java Advanced

Practice Java

GeeksforGeeks
Some Private Education Private Limited
Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305